

PROYECTO DE DESARROLLO DE SOLUCIONES TECNOLÓGICAS ASISTIDAS POR IA



Bitácora de prompts y desarrollo de fases.

Genesis Villalobos

Alanís Bordones

Francisca Trincado

Gabriela Rojas

CONTENIDO

FASE 1: DESCUBRIMIENTO Y PROTOTIPADO	3
NotebookLM	3
CREACIÓN DE DOCUMENTO TÉCNICO	16
Resumen NotebookLM	23
STITCH	23
FASE 2: DESARROLLO ASISTIDO POR IA	28
GoogleAI Studio	28
DESPLIEGUE	49

FASE 1: DESCUBRIMIENTO Y PROTOTIPADO

NotebookLM

Archivos entregados:

1. Descriptor de proyecto
2. Descriptor de negocio y problemática
3. Enlace a páginas con información de estrategias de *prompt engineering*.

Prompt 1:

Actúa como un Consultor Senior de Sistemas de Información y experto en Business Process Management (BPM).

Basándote única y exclusivamente en los documentos de origen cargados sobre el caso de la empresa, realiza un análisis profundo de requerimientos y optimización de procesos enfocado en la transformación digital.

Genera un informe estructurado que contenga los siguientes grandes bloques:

1. MAPEO DEL ESTADO ACTUAL (AS IS) Y PAIN POINTS:

- Identifica y numera los principales puntos de dolor (Pain Points), ineficiencias operativas, cuellos de botella y problemas de consistencia de datos que sufre la empresa actualmente.
- Vincula cada pain point con el impacto directo que tiene en el negocio (pérdida de clientes, costos altos, demoras, etc.).

2. REQUERIMIENTOS DE LA SOLUCIÓN (INGENIERÍA DE REQUISITOS):

- Clasifica los requerimientos necesarios para diseñar la nueva aplicación web/sistema en:
 - a. Requerimientos Funcionales (RF): Qué debe hacer el sistema explícitamente (ej. módulos, flujos de datos, automatizaciones clave, integraciones).
 - b. Requerimientos No Funcionales (RNF): Atributos de calidad del sistema relevantes para el contexto (ej. seguridad, escalabilidad, disponibilidad, interfaz responsive).

2. PROPUESTA DEL ESTADO OBJETIVO (TO BE):

- Describe brevemente cómo la introducción de la nueva solución tecnológica asistida por IA resolverá los pain points identificados y optimizará el flujo del proceso.

Mantén un tono profesional, técnico y ejecutivo. Estructura la respuesta utilizando títulos claros y listas con viñetas para que sea directamente trasladable al diseño de la base de datos SQL y al prototipado de interfaces.

Output 1:

INFORME DE CONSULTORÍA: TRANSFORMACIÓN DIGITAL Y OPTIMIZACIÓN DE PROCESOS - YOGA STUDIO "RESPIRA PROFUNDO"

Introducción El presente informe detalla el análisis de la situación actual y la hoja de ruta técnica para la digitalización de "Respira Profundo", el estudio de yoga de Valentina. Actualmente, el negocio opera bajo un modelo de gestión manual que limita su escalabilidad y afecta la experiencia del cliente.

1. MAPEO DEL ESTADO ACTUAL (AS IS) Y PAIN POINTS

A partir del análisis del caso, se identifican las siguientes ineficiencias críticas:

1. **Gestión de Reservas Descentralizada (vía Instagram):** El uso de redes sociales para reservas genera una carga administrativa alta y falta de sincronización en tiempo real.
 - **Impacto:** Demoras en respuestas y riesgo de sobreventa de cupos.
1. **Registro Manual de Asistencias y Saldos (Excel):** La actualización manual de "restar una clase cada vez que asiste" es propensa al error humano.
 - **Impacto:** **Inconsistencia de datos** y reclamos de clientes por cobros indebidos, lo que erosiona la confianza y puede causar **pérdida de clientes (churn)**.
1. **Opacidad en Métricas de Rendimiento:** No existe una forma ágil de identificar qué instructores atraen más alumnos fieles o cuáles horarios son los más rentables.
 - **Impacto:** Toma de decisiones basada en intuición y no en datos, lo que deriva en **costos de oportunidad** por horarios mal aprovechados.
1. **Falta de Autogestión para el Alumno:** Los clientes no tienen visibilidad de su saldo de clases ni pueden reservar de forma autónoma.
 - **Impacto:** Alta fricción en la experiencia del usuario y aumento de consultas operativas hacia Valentina.

2. REQUERIMIENTOS DE LA SOLUCIÓN (INGENIERÍA DE REQUISITOS)

Para solventar los problemas identificados y cumplir con la misión de automatización, se definen los siguientes requerimientos:

Requerimientos Funcionales (RF)

- **RF1 - Gestión de Membresías:** El sistema debe permitir la compra y asignación de paquetes (10 clases, 20 clases y mensualidad ilimitada).
- **RF2 - Registro de Asistencia Automatizado:** Módulo para que el instructor o el sistema descuente automáticamente una clase del saldo del alumno tras la validación de su asistencia.
- **RF3 - App de Autogestión (Cliente):** Interfaz para que los alumnos visualicen su saldo de clases restantes y realicen reservas en tiempo real.

- **RF4 - Sistema de Notificaciones:** Automatización de recordatorios de clases y alertas de vencimiento de paquetes.
- **RF5 - Dashboard Administrativo (Métricas):** Visualización de indicadores clave de desempeño (KPIs) que responda a:
 - Demanda por instructor y horario.
 - Estado del cliente (Activo, por vencer o "dormido").
 - Ingreso promedio por alumno.

Requerimientos No Funcionales (RNF)

- **RNF1 - Disponibilidad:** El sistema de reservas debe estar disponible 24/7 para garantizar la reserva en tiempo real.
- **RNF2 - Integridad de Datos:** Los cálculos de saldos deben ser exactos para eliminar los reclamos por "cobros indebidos".
- **RNF3 - Interfaz Responsive:** La solución debe ser accesible desde dispositivos móviles para los alumnos y en versión de escritorio para el dashboard de Valentina.
- **RNF4 - Seguridad de la Información:** Protección de los datos personales y registros de asistencia de los alumnos.

3. PROPUESTA DEL ESTADO OBJETIVO (TO BE)

La transformación hacia el estado **TO BE** se centra en la eliminación del "Caos Manual" mediante un ecosistema digital integrado.

La introducción de la nueva solución tecnológica, potenciada por una base de datos centralizada (SQL) y analítica de datos, permitirá:

- **Optimización del Flujo:** El proceso de reserva pasará de ser una interacción manual en Instagram a un flujo autoservicio en la app, reduciendo el tiempo de gestión en un 100% para Valentina.
- **Eliminación de Conflictos de Datos:** Al automatizar el descuento de clases y el registro de asistencia, se garantiza la transparencia absoluta hacia el cliente, eliminando las quejas por errores en el Excel.
- **Inteligencia de Negocio:** El dashboard convertirá los datos operativos en decisiones estratégicas, permitiendo a Valentina ajustar la oferta de clases (30 semanales) según la demanda real por instructor y horario, maximizando la rentabilidad del estudio.

Este enfoque no solo resuelve los problemas operativos inmediatos, sino que posiciona a **Respira Profundo** como un negocio moderno, escalable y centrado en la experiencia del cliente.

Prompt 2:

Usa el documento de desarrollo de soluciones tecnologicas para que definamos la batería de herramientas que utilizaremos. Tratemos de mantener una complejidad media. Comparemos lenguajes y herramientas para trabajar frontend y backend, considerando que la aplicación será realizada por herramientas de IA pero con revisión humana con conocimientos medios – bajos de programación. Teniendo esto en cuenta, recomiéndame lenguajes y herramientas.

Output 2:

*Como Consultor Senior, y basándome en los lineamientos del documento "**Desarrollo de Soluciones Tecnológicas Asistidas por IA**", he diseñado una propuesta de stack tecnológico de **complejidad media**. Esta selección busca el equilibrio entre potencia y facilidad de supervisión para un equipo con conocimientos de programación medios-bajos, apoyándose fuertemente en la generación de código vía IA.*

A continuación, presento la comparativa y la recomendación final de herramientas para el proyecto "Respira Profundo".

1. COMPARATIVA DE TECNOLOGÍAS (FRONTEND Y BACKEND)

De acuerdo con las opciones sugeridas en las fuentes, evaluamos las alternativas considerando que la aplicación será desarrollada por una IA y revisada por humanos:

Frontend: Interfaz de Usuario

Herramienta	Ventajas para el Perfil del Equipo	Desventajas / Complejidad
React	<i>Muy popular; la IA (Gemini/Claude) genera código muy preciso por la gran cantidad de ejemplos disponibles. Componentes fáciles de leer.</i>	<i>Requiere entender el manejo de "estados", lo cual puede ser un poco abstracto al inicio.</i>
Angular	<i>Estructura muy organizada y estricta, lo que evita errores de arquitectura.</i>	<i>Curva de aprendizaje más alta; el código generado es más extenso y difícil de auditar manualmente.</i>

- **Recomendación: React.** Es ideal para prototipado rápido y su sintaxis es más "legible" para una revisión humana rápida asistida por IA.

Backend: Lógica de Negocio y API

Herramienta	Ventajas para el Perfil del Equipo	Desventajas / Complejidad
Python (FastAPI)	<i>Sintaxis casi idéntica al lenguaje natural; muy fácil de revisar. Documentación automática (Swagger) que permite probar la lógica sin programar.</i>	<i>Ninguna crítica para este nivel de complejidad.</i>

Node.js (Express)	Permite usar el mismo lenguaje que en el frontend (JavaScript/TypeScript).	Puede volverse desordenado si la IA no sigue un patrón claro, dificultando la depuración.
--------------------------	--	---

- **Recomendación: Python con FastAPI.** Es el lenguaje por excelencia para trabajar con IA, y su claridad es insuperable para quienes tienen conocimientos básicos de programación.

2. BATERÍA DE HERRAMIENTAS RECOMENDADA (THE STACK)

Para cumplir con las fases de **Descubrimiento**, **Prototipado** e **Implementación** definidas en el proyecto, propongo el siguiente ecosistema:

A. Herramientas de Diseño e Ideación (Fase 1)

- **NotebookLM:** Para la gestión del conocimiento, síntesis de pain points y estructuración de la lógica de negocio basada en el caso de Valentina.
- **Stitch:** Fundamental para construir el prototipo interactivo (mockup) de alta fidelidad antes de escribir una sola línea de código.

B. Herramientas de Desarrollo y Programación (Fase 2)

- **Gemini / Google AI Studio:** Actuará como el **asistente de programación (pair-programming)** principal para generar los scripts de la base de datos (DDL/DML) y la lógica del servidor.
- **Frontend: React** (utilizando Vite para una configuración rápida).
- **Backend: Python con FastAPI** (por su simplicidad y velocidad).
- **Base de Datos: MySQL o PostgreSQL.** Ambas son bases de datos relacionales SQL estándar que permiten manejar los paquetes de clases y asistencias de forma robusta.

C. Gestión y Entrega

- **GitHub / GitLab:** Para el control de versiones y almacenamiento del código fuente en las carpetas requeridas (/docs, /db, /backend, /frontend).

3. JUSTIFICACIÓN DE LA CONSULTORÍA

Esta selección de herramientas minimiza el riesgo operativo por dos razones clave:

1. **Auditalidad:** El código de **Python** y **React** es el más fácil de "leer" para un humano que actúa como supervisor de la IA.
2. **Soporte de la IA:** Herramientas como **Gemini** tienen un rendimiento superior generando código en estos lenguajes debido a su amplia documentación global, lo que reduce la probabilidad de errores lógicos complejos en el sistema de reservas de "Respira Profundo".

Siguiente paso sugerido: Iniciar el prototipado en **Stitch** para definir visualmente cómo los alumnos verán su saldo de clases antes de proceder a la generación de código en Gemini.

Prompt 3:

Nace de la necesidad de asegurarnos de trabajar con herramientas / lenguajes que podamos entender para mejorar el flujo de desarrollo.

Es necesario que mantengamos las herramientas mas sencillas que puedan entregar un resultado aceptable en términos de calidad tanto frontend como backend. ¿Por eso creo que optaremos por FastAPI en el backend y en el frontend que recomiendas?

Output 3:

*Para mantener el equilibrio entre **sencillez, calidad y facilidad de desarrollo asistido por IA**, la recomendación definitiva para el frontend es **React**.*

A continuación, justifico esta elección como Consultor Senior, basándome en los requerimientos técnicos y los objetivos del proyecto:

Recomendación: React para el Frontend

*Aunque los documentos del proyecto mencionan tanto React como Angular, **React** es la opción más adecuada por las siguientes razones:*

- **Sinergia con la IA:** Debido a su enorme popularidad, modelos como Gemini o Claude generan código de React extremadamente preciso y funcional. Esto facilita que personas con conocimientos medios-bajos puedan supervisar y corregir el código generado.
- **Arquitectura Basada en Componentes:** El caso "Respira Profundo" requiere dos interfaces distintas: una **App para alumnos** (ver saldos y reservar) y un **Dashboard para Valentina** (métricas de negocio). React permite crear "piezas" de interfaz (componentes) reutilizables, simplificando la construcción de ambas interfaces sin duplicar esfuerzo.
- **Curva de Aprendizaje:** Comparado con Angular, React tiene una estructura menos rígida y más intuitiva, lo que reduce la frustración al momento de integrar el frontend con el backend de FastAPI que ya han seleccionado.

El Flujo de Trabajo Recomendado

Para asegurar un resultado de alta calidad con el menor esfuerzo técnico, deben seguir el proceso estipulado en las fases del proyecto:

1. **Diseño Previo en Stitch:** Antes de escribir código, utilicen **Stitch** para crear el prototipo interactivo (mockup). Esto les permitirá definir visualmente cómo se verán el dashboard y la app antes de pedirle a la IA que genere el código.
2. **Generación de Código Dirigida:** Una vez tengan el diseño en Stitch, usen Gemini o AI Studio para traducir esos diseños a componentes de React, asegurándose de documentar el proceso en su **Bitácora de Prompts**.

3. **Integración Sencilla:** La combinación **FastAPI + React** es un estándar de la industria. FastAPI entrega los datos en formato JSON de forma muy limpia, y React los consume con facilidad, lo que garantiza que los alumnos vean sus cupos en tiempo real, tal como lo pide la misión del grupo.

Conclusión: Mantenerse en el stack **FastAPI (Backend) + React (Frontend)** cumple con la premisa de complejidad media, asegura la entrega de un producto funcional para el 19 de junio y permite que la IA haga el "trabajo pesado" de programación mientras ustedes actúan como supervisores del flujo de negocio.

Prompt 4:

Para la base de datos es necesario que pueda desplegarse online, no usar algo local, con una escalabilidad media, define la mejor opción a utilizar en el contexto del negocio.

Output 4:

*Como Consultor Senior, y atendiendo a tu requerimiento de contar con una infraestructura **online, no local y con escalabilidad media**, es fundamental seleccionar un servicio que se integre perfectamente con el stack ya definido (**FastAPI + React**) y que cumpla con el requisito de ser una **base de datos SQL relacional**.*

Para el caso de "Respira Profundo", donde necesitamos gestionar paquetes de clases, registros de asistencia y saldos de alumnos de forma íntegra, te recomiendo las siguientes opciones de despliegue en la nube:

1. PostgreSQL en Supabase o Neon (Recomendación Principal)

*Aunque las fuentes mencionan tanto MySQL como PostgreSQL, para un entorno de **complejidad media** asistido por IA, **PostgreSQL** es altamente recomendable.*

- **Por qué utilizarlo:** Ofrecen planes gratuitos robustos y escalabilidad automática (especialmente Neon).
- **Ventaja Técnica:** Se integran de forma nativa con Python y FastAPI. **Supabase**, en particular, proporciona una interfaz web muy intuitiva que facilita la visualización de las tablas de "Membresías" y "Reservas" sin necesidad de herramientas externas complejas.
- **Entrega:** Facilita la exportación del **script .sql** necesario para la carpeta /db del repositorio final.

2. MySQL en Railway o PlanetScale

*Si prefieres optar por **MySQL**, que es otra de las opciones sugeridas en los requerimientos del proyecto:*

- **Por qué utilizarlo:** **Railway** permite desplegar tanto tu base de datos MySQL como tu servidor FastAPI en el mismo entorno de nube de forma simplificada.
- **Escalabilidad:** **PlanetScale** es conocido por su capacidad de escalabilidad media-alta, permitiendo modificar el esquema de la base de datos (como agregar nuevas métricas para el dashboard de Valentina) sin tiempos de inactividad.

3. Justificación de la Elección para "Respira Profundo"

Considerando que el objetivo es pasar "del caos manual a la solución digital", una base de datos online garantiza que:

- **Disponibilidad Real:** Los alumnos podrán reservar su cupo en tiempo real desde cualquier dispositivo, cumpliendo con la **Misión del Grupo**.
- **Consistencia de Datos:** Al no ser local, evitamos el riesgo de pérdida de información que Valentina sufría con su Excel manual.

- **Facilidad para la IA:** Herramientas como Gemini pueden generar scripts **DDL y DML** optimizados específicamente para PostgreSQL o MySQL, asegurando que la estructura de tablas soporte las consultas de negocio requeridas (ej. ingresos promedio por alumno o demanda por instructor).

Mi consejo experto: Opta por **PostgreSQL hospedado en Supabase**. Es la opción que mejor equilibra la facilidad de uso para un equipo con conocimientos medios de programación y la potencia necesaria para escalar el negocio de Valentina de manera profesional. Recuerda que, independientemente del proveedor, debes incluir el script de creación de tablas en la carpeta /db de tu repositorio.

Prompt 5:

Definamos un tipo de diseño de interfaz que se vea moderna sin mucha complejidad en código, ya sea glassformismo, minimalismo, etc.

Output 5:

Como Consultor Senior, y considerando que el proyecto de **"Respira Profundo"** debe combinar una estética profesional con una implementación técnica ágil asistida por IA, mi recomendación definitiva es optar por un estilo de **Minimalismo Funcional (Clean Design)** con toques de **Soft UI**.

A continuación, detallo la propuesta de diseño, justificando por qué es la mejor opción para reducir la complejidad del código sin sacrificar la modernidad:

1. Concepto Visual: "Minimalismo Zen"

Dado que el negocio es un estudio de yoga, la interfaz debe transmitir paz, orden y claridad, valores que coinciden con los objetivos del minimalismo. Este estilo evita el exceso de elementos decorativos, enfocándose en el contenido y la facilidad de uso.

- **Paleta de Colores:** Uso de tonos neutros (blancos, grises suaves) combinados con colores de acento orgánicos (verdes suaves o el azul/púrpura que ya se aprecia en la identidad visual del caso).
- **Tipografía:** Sans-serif moderna (ej. Inter o Roboto) que garantiza legibilidad tanto en la app del alumno como en el dashboard administrativo.

2. Por qué Minimalismo sobre Glassmorfismo o Neumorfismo

- **Complejidad de Código:** El glassmorfismo requiere el manejo de propiedades CSS complejas como `backdrop-filter` y transparencias que pueden dar problemas de accesibilidad o rendimiento si no se aplican bien. El minimalismo usa principalmente **espaciado (padding/margin) y tipografía**, lo cual es extremadamente sencillo de generar y revisar mediante IA.
- **Sinergia con la IA:** Los modelos de lenguaje como Gemini generan código de **Tailwind CSS** (un framework de diseño) con mucha mayor precisión cuando se les pide un estilo limpio y basado en tarjetas (cards), lo que acelera la fase de implementación.
- **Mantenibilidad:** Para un equipo con conocimientos medios-bajos de programación, es mucho más fácil depurar un diseño minimalista basado en cajas claras que uno con múltiples capas de sombras y efectos de cristal.

3. Elementos Clave en el Frontend (React)

Para la aplicación de Valentina, el diseño minimalista se traducirá en los siguientes componentes:

- **Cards de Información:** Tarjetas simples con bordes redondeados suaves para mostrar el "Saldo de Clases" o los "Horarios de Clase".

- **Uso del Espacio en Blanco:** Fundamental para que la interfaz "respire", permitiendo que el usuario encuentre rápidamente el botón de "Reservar Cupo".
- **Iconografía Lineal:** Iconos finos y sencillos para las acciones principales, evitando saturación visual.

4. Flujo de Trabajo con Herramientas

1. **Stitch (Prototipado):** Deben utilizar **Stitch** para definir este estilo minimalista en el mockup interactivo. Al ser un diseño de "bloques limpios", el paso del prototipo al código será casi directo.
2. **Generación de Componentes:** Al pedirle código a la IA, el prompt debe ser específico: "Genera un componente de React usando Tailwind CSS con un diseño minimalista, bordes suaves, mucho espacio en blanco y una paleta de colores pastel para un estudio de yoga".

Conclusión del Consultor: El **Minimalismo Funcional** es la opción más estratégica. Asegura una experiencia de usuario (UX) premium para los alumnos de Valentina y un código fuente limpio y fácil de mantener para el equipo de desarrollo, cumpliendo con la entrega del repositorio y el video pitch para el 19 de junio.

Prompt 6:

Añadamos una sección con parámetros de patrones de diseño comunes en aplicaciones exitosas con buena UX/UI. Realiza una investigación sobre patrones y sistemas de diseño exitosos.

Output 6:

PATRONES Y SISTEMAS DE DISEÑO DE ÉXITO (UX/UI)

Para transformar el "caos manual" en una solución digital profesional, la aplicación implementará patrones de diseño validados por la industria.

Nota: La siguiente información sobre sistemas de diseño específicos (como Material Design o Ant Design) no se encuentra explícitamente en las fuentes proporcionadas y constituye una recomendación basada en estándares de la industria que usted puede verificar independientemente.

A. Patrones de Interfaz (UI Patterns) para el Alumno

Considerando que los alumnos necesitan reservar cupos en tiempo real, se aplicarán los siguientes patrones:

- **Card-Based Layout (Diseño basado en tarjetas):** Se utilizarán tarjetas para mostrar los diferentes paquetes (10 clases, 20 clases, ilimitado). Este patrón facilita la lectura rápida y es altamente responsive en dispositivos móviles.
- **Skeleton Screens:** Durante la carga de la disponibilidad de clases, se mostrarán "esqueletos" de contenido. Esto reduce la percepción de tiempo de espera, mejorando la satisfacción del usuario.

B. Patrones de Dashboard para la Administración (Valentina)

El dashboard debe responder a preguntas críticas de negocio de forma inmediata:

- **F-Pattern Reading:** La disposición de las métricas (KPIs de demanda e ingresos) seguirá un patrón de lectura en "F", colocando la información más crítica en la parte superior izquierda.
- **Data Visualization Widgets:** Uso de gráficos de barras para la demanda por instructor y gráficos de velocímetro (gauges) para el estado de clientes activos/dormidos.
- **Empty States (Estados Vacíos):** Diseños amigables para cuando no hay datos (ej. "Aún no hay reservas para este horario"), evitando que la interfaz se vea rota o confusa.

C. Patrones de Arquitectura de Software (Backend/Frontend)

*Para asegurar que la integración entre **FastAPI** y **React** sea exitosa según las fases del proyecto:*

- **Separación de Preocupaciones (SoC):** El frontend (React) se encargará exclusivamente de la vista, mientras que el backend (FastAPI) manejará la lógica de negocio y la persistencia en SQL.

- **RESTful API:** La comunicación se realizará mediante servicios REST, asegurando que el intercambio de datos (como el saldo de clases) sea ligero y estandarizado.

D. Sistemas de Diseño de Referencia

Para el prototipado en **Stitch**, nos basaremos en los principios de:

- **Material Design (Google):** Por su enfoque en la jerarquía visual y el uso de sombras suaves, lo cual facilita la implementación de nuestro estilo de **Minimalismo Zen**.
- **Ant Design:** Excelente referencia para el dashboard administrativo, ya que ofrece componentes pre-diseñados para tablas y gráficos de métricas complejas.

Justificación de la Consultoría: La implementación de estos patrones de diseño asegura que la aplicación no solo sea funcional, sino que sea **intuitiva y fácil de usar**. Esto es vital para que Valentina pueda transitar del Excel a la plataforma digital sin fricciones y para que los alumnos perciban el valor inmediato de poder autogestionar sus clases en tiempo real. La combinación de estos patrones con el desarrollo asistido por IA permitirá alcanzar estos estándares de calidad antes del **19 de junio de 2026**

CREACIÓN DE DOCUMENTO TÉCNICO

Prompt:

Con todo esto en consideración, procedamos a crear el documento técnico de la aplicación asegurando considerar las herramientas que hemos mencionado y teniendo en cuenta lo siguiente:

- Mantener un enfoque ágil
- Documentar la arquitectura de sistema
- Documentar documentar posibles periodos de desarrollo
- Documentar UI/UX como un sistema moderno, aplicando minimalismo funcional
- Documentar patrones de diseño de UX/UI

Output:

*Este documento técnico constituye la hoja de ruta para la ejecución del proyecto "**Respira Profundo**", transformando la gestión manual actual en una solución digital escalable, moderna y eficiente. El enfoque se centra en la velocidad de entrega mediante el uso de **Inteligencia Artificial Generativa** como asistente de desarrollo.*

ESPECIFICACIÓN TÉCNICA Y PLAN DE DESARROLLO: RESPIRA PROFUNDO

1. ARQUITECTURA DEL SISTEMA

Se ha diseñado una arquitectura de tres capas, optimizada para un despliegue rápido y una alta cohesión entre componentes:

- **Capa de Presentación (Frontend):** Desarrollada en **React**. Se encargará de las interfaces para alumnos (reservas y saldos) y el dashboard de métricas para Valentina.
 - **Capa de Lógica de Negocio (Backend):** Implementada con **Python y FastAPI**. Gestionará la lógica de autenticación, el descuento automático de clases y el procesamiento de KPIs (Key Performance Indicators).
 - **Capa de Datos (Base de Datos):** **PostgreSQL** alojado en **Supabase**. Esta elección garantiza una base de datos SQL relacional online, eliminando la dependencia de archivos locales y asegurando la integridad de los datos de membresías.
 - **Capa de Integración de IA:** Uso de **Gemini / AI Studio** para el pair-programming, generación de scripts DDL/DML y refactorización de código.
-

2. PLANIFICACIÓN ÁGIL (SPRINTS DE DESARROLLO)

*Dado que la fecha límite es el **19 de junio de 2026**, se propone un cronograma de ejecución acelerada dividido en tres hitos críticos:*

- **Fase 1: Descubrimiento y Diseño (Días 1-2):**

- Sintetizar requerimientos en NotebookLM.
 - Crear prototipo interactivo (mockup) en **Stitch** siguiendo el flujo de navegación principal.
 - **Fase 2: Implementación Core y Backend (Días 3-4):**
 - Generar y ejecutar scripts **.sql** en Supabase (Carpeta **/db**).
 - Desarrollar la API en FastAPI para la gestión de paquetes y asistencias (Carpeta **/backend**).
 - **Fase 3: Frontend, Integración y Cierre (Día 5):**
 - Construir la interfaz en React basada en los diseños de **Stitch** (Carpeta **/frontend**).
 - Documentación de la **Bitácora de Prompts** y grabación del **Video Pitch** de 5 minutos.
-

3. DISEÑO UI/UX: MINIMALISMO FUNCIONAL (ZEN)

Para alinearse con la naturaleza del estudio de yoga y garantizar una implementación técnica sencilla pero de alta calidad, el sistema adoptará un estilo **Minimalismo Funcional**:

- **Filosofía Visual:** Interfaces despejadas con un enfoque en la "respiración" del diseño (espacios en blanco generosos). Esto facilita que el alumno encuentre su saldo y reserve su cupo sin distracciones.
 - **Soft UI & Cards:** Uso de tarjetas con bordes redondeados y sombras suaves para separar visualmente las clases disponibles de las métricas de negocio.
 - **Accesibilidad y Responsividad:** El diseño será mobile-first, asegurando que los alumnos puedan reservar desde sus teléfonos, cumpliendo con la misión del grupo de "reserva en tiempo real".
 - **Herramienta de Control:** Toda la experiencia de usuario será validada previamente en **Stitch** antes de proceder a la generación de código, minimizando errores de flujo.
-

4. ESTRATEGIA DE DESARROLLO ASISTIDO POR IA

El éxito de este documento técnico reside en la **Gestión de Prompts**:

- **Iteración:** El equipo no solo pedirá código, sino que utilizará la IA para corregir errores y optimizar la lógica de negocio compleja (ej. el cálculo automático de alumnos fieles o rentabilidad por horario).
- **Bitácora:** Se mantendrá un registro detallado en la carpeta **/docs** que evidencie cómo se refinaron las instrucciones dadas a Gemini para alcanzar los requerimientos funcionales.

*Este enfoque asegura que para el **19 de junio**, Valentina cuente con un sistema profesional que elimine el caos del Excel y los reclamos por cobros indebidos, permitiéndole tomar decisiones basadas en datos reales.*

Prompt 2:

Agreguemos una estrategia de prompt engineering acorde a tareas de desarrollo de software. Utiliza el siguiente enlace para basarte: <https://www.ibm.com/es-es/think/topics/prompt-engineering-techniques>.

Output 2:

ESPECIFICACIÓN TÉCNICA: SOLUCIÓN DIGITAL "RESPIRA PROFUNDO"

Este documento técnico establece la arquitectura, el diseño y la hoja de ruta para la implementación de la plataforma de gestión del estudio de yoga "Respira Profundo". El objetivo principal es migrar del "caos manual" de registros en Excel y reservas por Instagram hacia una solución automatizada y escalable.

1. ENFOQUE ÁGIL Y PERIODOS DE DESARROLLO

*La ejecución del proyecto se basa en un enfoque de **desarrollo ágil asistido por IA**, diseñado para cumplir con la entrega final del **19 de junio de 2026**. El cronograma se divide en dos fases principales:*

- **Fase 1: Descubrimiento y Prototipado (Ideación):**
 - **Actividades:** Síntesis de pain points con NotebookLM, definición de requerimientos funcionales y diseño del flujo de navegación interactivo.
 - **Herramienta de diseño:** **Stitch** para la creación de mockups de alta fidelidad.
- **Fase 2: Desarrollo Asistido por IA (Implementación):**
 - **Actividades:** Generación de scripts de base de datos (DDL/DML), desarrollo del Backend para la lógica de negocio y creación del Frontend basado en los diseños previos.
 - **Metodología:** Pair-programming utilizando **Gemini o Google AI Studio** como asistentes para acelerar la codificación.

2. ARQUITECTURA DEL SISTEMA

Se ha seleccionado una arquitectura de tres capas que asegura la separación de preocupaciones y facilita el mantenimiento:

- **Front-end (Capa de Presentación):** Implementado en **React**. Se encargará de la interfaz de autogestión para alumnos y el dashboard administrativo para Valentina.
- **Back-end (Capa de Lógica):** Desarrollado en **Python con FastAPI**. Gestionará la conexión con la base de datos, la lógica de descuento automático de clases y la generación de métricas.

- **Base de Datos (Capa de Datos): PostgreSQL (SQL Relacional)** desplegado en la nube (**Supabase**). Almacenará la información de usuarios, paquetes de clases (10, 20 o ilimitadas) y registros de asistencia.

3. UI/UX: MINIMALISMO FUNCIONAL

Para reflejar la esencia del estudio de yoga y garantizar una curva de aprendizaje mínima, se aplicará un sistema de **Minimalismo Funcional (Zen)**:

- **Simplicidad Visual:** Interfaces despejadas con un uso estratégico del espacio en blanco para reducir la carga cognitiva.
- **Claridad Operativa:** El flujo principal permitirá que el alumno vea cuántas clases le quedan y reserve su cupo en tiempo real con la menor cantidad de clics posible.
- **Estética Moderna:** Uso de tipografías legibles y una paleta de colores suaves que transmitan profesionalismo y calma.

4. PATRONES DE DISEÑO DE UX/UI

Se implementarán patrones validados para asegurar una experiencia de usuario exitosa:

- **Card-Based Layout:** Uso de tarjetas para presentar los horarios de las 30 clases semanales y los detalles de los paquetes.
- **Dashboard de Control:** Un panel administrativo diseñado para que Valentina obtenga respuestas inmediatas sobre la demanda por instructor, el estado de los clientes (activos, por vencer o dormidos) y el ingreso promedio por alumno.
- **Feedback en Tiempo Real:** Notificaciones y confirmaciones visuales inmediatas tras realizar una reserva o registrar una asistencia, eliminando la incertidumbre del sistema manual anterior.

5. ESTRATEGIA DE PROMPT ENGINEERING

El desarrollo de software se documentará en una **Bitácora de Prompts**, detallando cómo se utilizaron instrucciones iterativas para:

- Generar el esquema SQL funcional para el seguimiento de membresías.
- Refactorizar código y corregir errores en la integración Front-end/Back-end.
- Implementar la lógica compleja necesaria para los reportes de rentabilidad por horario.

6. ESTRUCTURA DE ENTREGABLES (REPOSITORIO)

De acuerdo con los requerimientos del proyecto, la solución se organizará de la siguiente manera:

- **/docs:** Informe ejecutivo, enlace al prototipo en Stitch y Bitácora de Prompts.
- **/db:** Scripts SQL de estructura y datos de prueba.
- **/backend:** Código fuente del servidor (FastAPI).

- ***/frontend:*** Código fuente de la interfaz (React).
- ***Video Pitch:*** Demostración funcional de 5 minutos que valide la solución al problema de Valentina.

Prompt 3:

Explayemos un poco más la documentación para ser utilizada como input en los modelos de IA generadores del código. Entreguemos un poco más de detalle técnico

Output 3:

*Este anexo técnico está diseñado para servir como el **Contexto Maestro** que deberás entregar a los modelos de IA (Gemini, Claude o AI Studio) para la generación del código. Se profundiza en la estructura de datos, la lógica de negocio y las especificaciones de la API, asegurando que la IA produzca resultados funcionales alineados con el caso de "Respira Profundo".*

ESPECIFICACIONES TÉCNICAS PARA GENERACIÓN DE CÓDIGO (INPUT IA)

1. ARQUITECTURA DE BASE DE DATOS (ESQUEMA SQL)

Para resolver la inconsistencia de datos y los reclamos por cobros indebidos, el modelo debe ser estrictamente relacional.

Entidades y Atributos Sugeridos:

- **users:** id (PK), nombre, email, rol ('alumno', 'instructor', 'admin').
- **packages:** id (PK), nombre ('10 clases', '20 clases', 'ilimitado'), precio, cantidad_clases (entero, usar 999 para ilimitado).
- **memberships:** id (PK), user_id (FK), package_id (FK), clases_restantes (entero), fecha_vencimiento, estado ('activo', 'vencido').
- **class_schedule:** id (PK), instructor_id (FK), dia_semana, hora_inicio, cupo_maximo (por defecto 30 clases semanales disponibles en el estudio).
- **bookings:** id (PK), user_id (FK), schedule_id (FK), fecha_clase, asistencia (booleano, por defecto False).

Regla de Negocio Técnica: La IA debe generar un **Trigger** o lógica de aplicación que, al marcar asistencia = True en bookings, reste automáticamente -1 en memberships.clases_restantes, a menos que el paquete sea 'ilimitado'.

2. CONTRATO DE API (BACKEND FASTAPI)

Para el desarrollo del backend, la IA debe generar los siguientes endpoints clave:

Módulo de Alumno (App):

- **GET /me/status:** Devuelve el saldo de clases y fecha de vencimiento del alumno autenticado.
- **GET /classes/available:** Lista las clases con cupos disponibles filtrando por día.
- **POST /bookings/reserve:** Crea un registro en la tabla bookings validando que el alumno tenga saldo > 0.

Módulo Administrativo (Dashboard de Valentina):

- *GET /admin/stats/demand: Agrega datos para responder: ¿Qué instructor y horario tienen más demanda?*
 - *GET /admin/stats/clients: Filtra alumnos por clases_restantes < 3 (por vencer) o sin reservas en 15 días (dormidos).*
 - *GET /admin/stats/revenue: Calcula el ingreso promedio por alumno (Total ventas / Total alumnos).*
-

3. ESPECIFICACIONES DE FRONTEND (REACT + TAILWIND)

Para lograr el "Minimalismo Zen", los prompts deben instruir a la IA sobre la estructura de componentes:

- **Componente BalanceCard:** Debe mostrar el número de clases restantes en un círculo grande centralizado, usando colores suaves (ej. #F3F4F6 de fondo y #10B981 para el texto si es positivo).
 - **Componente ClassCalendar:** Una lista vertical de tarjetas simples. Cada tarjeta debe tener el nombre del instructor, la hora y un botón de "Reservar" que cambie de estado visualmente tras la acción.
 - **Dashboard de Valentina:** Utilizar una rejilla (Grid) de 3 columnas para los KPIs principales, con gráficos de barras simples generados mediante librerías livianas como Recharts.
-

4. LÓGICA DE NEGOCIO PARA EL PROMPT (REGLAS DE ORO)

Al interactuar con la IA para generar el código, asegúrate de incluir estas instrucciones lógicas:

1. **Validación de Cupos:** "Al realizar una reserva, el sistema debe verificar primero si el número de registros en bookings para ese schedule_id es menor al cupo_maximo".
 2. **Manejo de Paquetes Ilimitados:** "Si package.nombre contiene 'ilimitado', la función de descuento de clases no debe restar saldo, pero sí registrar la asistencia".
 3. **Seguridad:** "Utiliza JWT (JSON Web Tokens) para asegurar que un alumno no pueda ver el saldo de otro ni acceder al dashboard de Valentina".
-

5. ESTRUCTURA DE CARPETAS DEL REPOSITORIO

Instruye a la IA para que el código generado se organice según los requerimientos del proyecto:

- /db: Archivo init.sql con el DDL completo.

- */backend: Estructura con main.py, models.py, schemas.py y database.py.*
- */frontend: Carpeta src/components con archivos .jsx separados para una revisión humana más sencilla.*

Este nivel de detalle técnico permitirá que la IA actúe como un verdadero "Pair-Programmer", entregando código que Valentina pueda utilizar inmediatamente para eliminar sus problemas de cobros indebidos y falta de métricas.

Resumen NotebookLM

Categoría	Detalle y Definición Técnica
Contexto del Caso	Estudio de yoga con 4 instructores y 30 clases semanales; transita de un sistema manual (Excel/Instagram) a una solución digital.
Misión del Proyecto	Crear un sistema para gestionar paquetes, registrar asistencias automáticamente, enviar recordatorios y generar métricas de negocio.
Stack Tecnológico	Frontend: React. Backend: Python/FastAPI. Base de Datos: PostgreSQL (Online en Supabase).
Requerimientos Funcionales	Autogestión de alumnos (reservas y saldos), automatización de cobros/asistencias y dashboard de KPIs (demandas, fidelización, ingresos).
Filosofía de Diseño	Minimalismo Zen (Funcional): Estética limpia, uso de espacios en blanco, tipografía legible y diseño basado en tarjetas (<i>cards</i>).
Estrategia de IA	Prompt Engineering: Uso de <i>Role Prompting</i> , <i>Chain-of-Thought</i> y <i>Few-shot</i> para generación y depuración de código.
Patrones de UX/UI	<i>Card-Based Layout</i> , <i>F-Pattern Reading</i> para el dashboard administrativo y navegación móvil intuitiva para alumnos.
Fases de Desarrollo	Fase 1: Prototipado en Stitch. Fase 2: Implementación de código (DDL, Backend, Frontend) asistida por Gemini/AI Studio.
Entregables Obligatorios	Repositorio con carpetas /docs (informe, bitácora), /db (scripts SQL), /backend, /frontend y un Video Pitch de 5 min.
Fecha Límite	19 de junio de 2026 (Entrega improrrogable vía Campus Virtual).

Este resumen actúa como la **línea base** para que el equipo comience la generación de los componentes técnicos, asegurando que cada línea de código escrita por la IA responda directamente a los problemas operativos identificados en el estudio de Valentina.

STITCH

Prompt 1:

Rol: Actúa como un **Diseñador Senior de UI/UX** especializado en bienestar y sistemas de gestión empresarial. Tu objetivo es diseñar un prototipo de alta fidelidad para "Respira Profundo", un estudio de yoga que busca automatizar su flujo de reservas y métricas.

Estilo Visual (Minimalismo Zen):

- **Estética:** Aplica un diseño de **Minimalismo Funcional** con mucho espacio en blanco para que la interfaz "respire".
- **Paleta de Colores:** Utiliza **tonos pasteles suaves** (verde salvia, lavanda pálido y blanco hueso) para transmitir calma y profesionalismo.
- **Componentes:** Implementa **Soft UI** con tarjetas (*cards*) de bordes muy redondeados y sombras sutiles para organizar la información.

Arquitectura del Prototipo (Chain-of-Thought): Diseña un flujo interactivo que contenga dos vistas principales basadas en los requerimientos del negocio:

1. Vista del Alumno (App Móvil):

- **Pantalla de Inicio:** Muestra una tarjeta destacada con el **saldo de clases restantes** del alumno (ej. "Te quedan 4 clases de tu paquete de 10").
- **Calendario de Reservas:** Una lista vertical de las **30 clases semanales** disponibles. Cada clase debe mostrar: Nombre de la clase, Instructor (de los 4 disponibles), Hora y un botón claro de "Reservar Cupo".
- **Navegación:** Barra inferior (*Bottom Nav*) con iconos lineales para: Inicio, Calendario, Mi Perfil.

2. Vista de Administración / Valentina (Dashboard Desktop):

- **Panel de Métricas (KPIs):** Diseña tres tarjetas superiores que respondan a:
 1. **Demanda:** Gráfico simple de los horarios e instructores más concurridos.
 2. **Estado de Clientes:** Indicador visual de alumnos "Activos", "Por Vencer" y "Dormidos".
 3. **Rentabilidad:** El valor del "Ingreso promedio por alumno".
- **Gestión de Clases:** Una tabla limpia que permita a Valentina marcar la asistencia de los alumnos para que el sistema descuente automáticamente la clase del saldo.

Interacciones Clave:

- Al hacer clic en "Reservar" en la vista del alumno, debe mostrarse una confirmación visual instantánea.
- Asegura que el diseño sea **responsive**, priorizando la facilidad de uso para los alumnos que reservan desde Instagram o dispositivos móviles.

Output 1:

+ New Class

Dashboard

Schedules

Students

Attendance

Finances

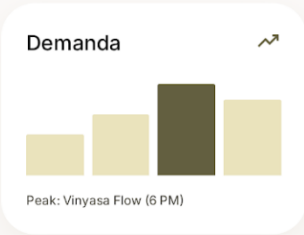
Help

Logout

Admin User
Manager

Overview

Welcome back to the studio. Here is today's snapshot.



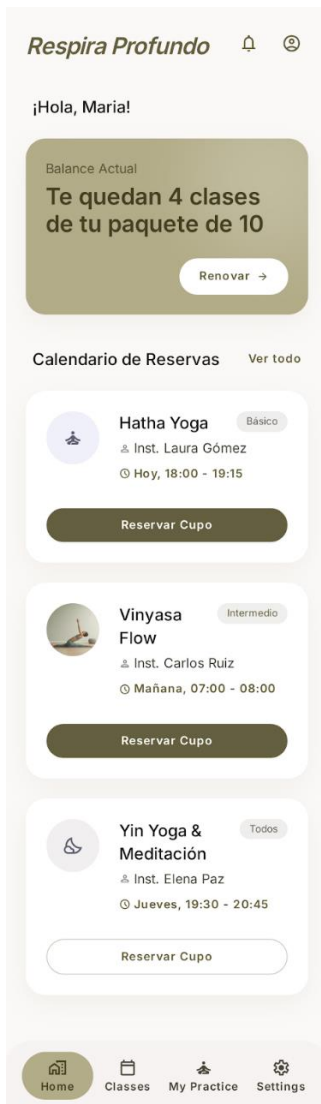
Gestión de Clases

Today's schedule and attendance.

Search student or class...

Class & Time	Instructor	Level	Capacity	Action
Vinyasa Flow 08:00 AM - 09:15 AM	M Mariana	All Levels	18 / 20	Mark Attendance
Hatha Yoga 10:00 AM - 11:30 AM	C Carlos	Beginner	12 / 15	Mark Attendance
Ashtanga Full Primary 06:00 PM - 07:30 PM	S Sofia	Advanced	20 / 20	Full / Waitlist

View Full Schedule



Los colores y tipografías no eran las esperadas. Se utilizaron paleta de colores generadas en la página web

[Coolors - The super fast color palettes generator!](#)

- #F9F8F8
- #CDD3CE
- #BBB5BD
- #AA6DA3
- #B118C8

Prompt 2:

#F9F8F8

#CDD3CE

#BBB5BD

#AA6DA3

#B118C8

Utiliza la paleta de colores entregada. Evita las letras cursivas y trata de mantener la tipografía mas soft y menos rígida

Output 2:

Respira Profundo

Admin Management

+ New Class

Dashboard

Schedules

Students

Attendance

Finances

Help

Logout

Admin User Manager

Overview

Welcome back to the studio. Here is today's snapshot.

Demanda

Peak: Vinyasa Flow (6 PM)

Estado de Clientes

Activos142

Por Vencer28

Dormidos15

Rentabilidad

Avg. Revenue / Student

\$85

+5.2% this month

Gestión de Clases

Today's schedule and attendance.

Search student or class...

Class & Time	Instructor	Level	Capacity	Action
Vinyasa Flow 08:00 AM - 09:15 AM	M Mariana	All Levels	18 / 20	Mark Attendance
Hatha Yoga 10:00 AM - 11:30 AM	C Carlos	Beginner	12 / 15	Mark Attendance
Ashtanga Full Primary 06:00 PM - 07:30 PM	S Sofia	Advanced	20 / 20	Full / Waitlist

View Full Schedule

27

FASE 2: DESARROLLO ASISTIDO POR IA

GoogleAI Studio

Archivos entregados:

1. .docx con documentación técnica, contexto y reglas de negocio de Respira Profundo
2. Archivos exportados desde Stitch (code.html)
3. Prompt con estrategia definida

Prompt 1:

Rol: Actúa como un Senior Full Stack Developer y experto en Arquitectura de Software. Tu objetivo es ayudarme a desarrollar una solución digital para "Respira Profundo", un estudio de yoga que actualmente sufre de ineficiencias operativas por el uso de procesos manuales

Contexto del Negocio: El estudio tiene 4 instructores y ofrece 30 clases semanales. Actualmente usan Excel y reservas por Instagram, lo que genera errores en los saldos de los alumnos y falta de métricas de negocio

. Necesitamos automatizar la gestión de paquetes (10, 20 clases e ilimitados), el registro de asistencias y la generación de un dashboard administrativo

Instrucciones de Diseño (Referencia Stitch): Te he proporcionado los archivos HTML generados en Stitch que contienen el prototipo visual.

Analiza la estructura de estos archivos para identificar los componentes de la interfaz.

Traduce estos diseños a componentes funcionales de React, utilizando Tailwind CSS.

Mantén el estilo de Minimalismo Zen: paleta de colores pasteles suaves, bordes redondeados (soft UI) y amplio uso de espacios en blanco.

Stack Tecnológico Requerido:

- Backend: Python con FastAPI.
- Frontend: React (usando Vite).
- Base de Datos: PostgreSQL (SQL Relacional).

Tareas de Desarrollo (Generar código por fases):

FASE 1: Base de Datos (Carpeta /db)

Genera el script SQL (DDL y DML) con las tablas: users, packages, memberships, class_schedule y bookings.

Incluye lógica para que, al marcar asistencia en bookings, se descuente automáticamente una clase del saldo en memberships (si el paquete no es ilimitado).

Proporciona datos de prueba para los 4 instructores y las 30 clases semanales.

FASE 2: Backend API (Carpeta /backend)

Crea una API REST con FastAPI que incluya endpoints para:

GET /me/status: Saldo y vencimiento del alumno.

POST /bookings/reserve: Reservar cupo validando saldo y capacidad máxima (30 cupos).

GET /admin/stats: Dashboard para Valentina que responda: ¿Qué instructor/horario tiene más demanda?, ¿Cuántos clientes están activos/dormidos?, y ¿Cuál es el ingreso promedio por alumno?

.

FASE 3: Frontend (Carpeta /frontend)

Desarrolla los componentes en React basados en los HTML de Stitch:

BalanceCard: Visualización clara del saldo del alumno.

BookingCalendar: Lista de clases con botón de reserva en tiempo real.

AdminDashboard: Visualización de los KPIs solicitados para Valentina mediante tablas y gráficos simples.

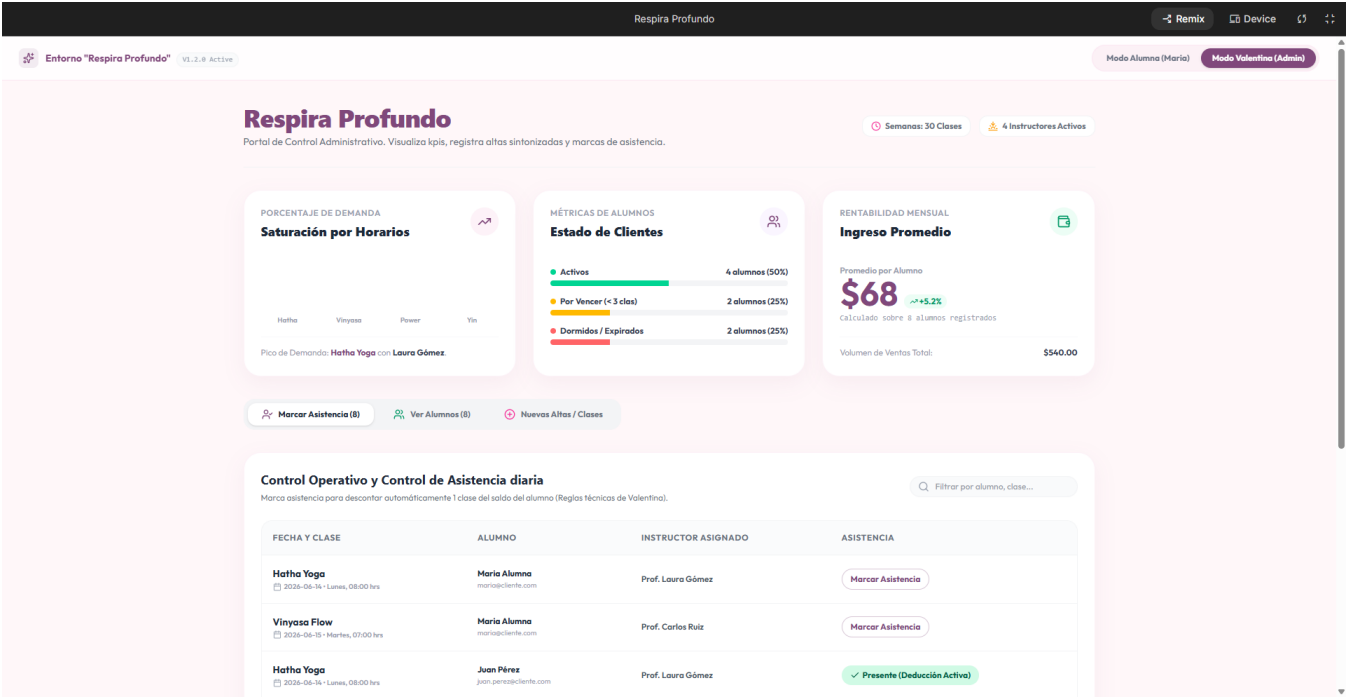
Restricciones de Calidad:

El código debe ser modular, limpio y fácil de revisar por un humano con conocimientos medios de programación.

Utiliza una estrategia de "Chain-of-Thought": antes de escribir el código, explica brevemente la lógica que seguirás para asegurar que se cumplen las reglas de negocio de Valentina

.

Output 1:



Creemos que es necesario un *sidebar* con opciones y que además exista un *login* para definir los ingresos de los usuarios y mostrar solo lo que corresponde a su rol.

Prompt 2:

Necesito que tenga un login que se conecte a la tabla de usuarios y permita visualizar solo la información correcta respecto al tipo de usuario. Además añade un sidebar para navegar entre vistas de forma mas intuitiva

Output 2:

Respira Profundo
YOGA & MEDITACIÓN STUDIO

CORREO ELECTRÓNICO

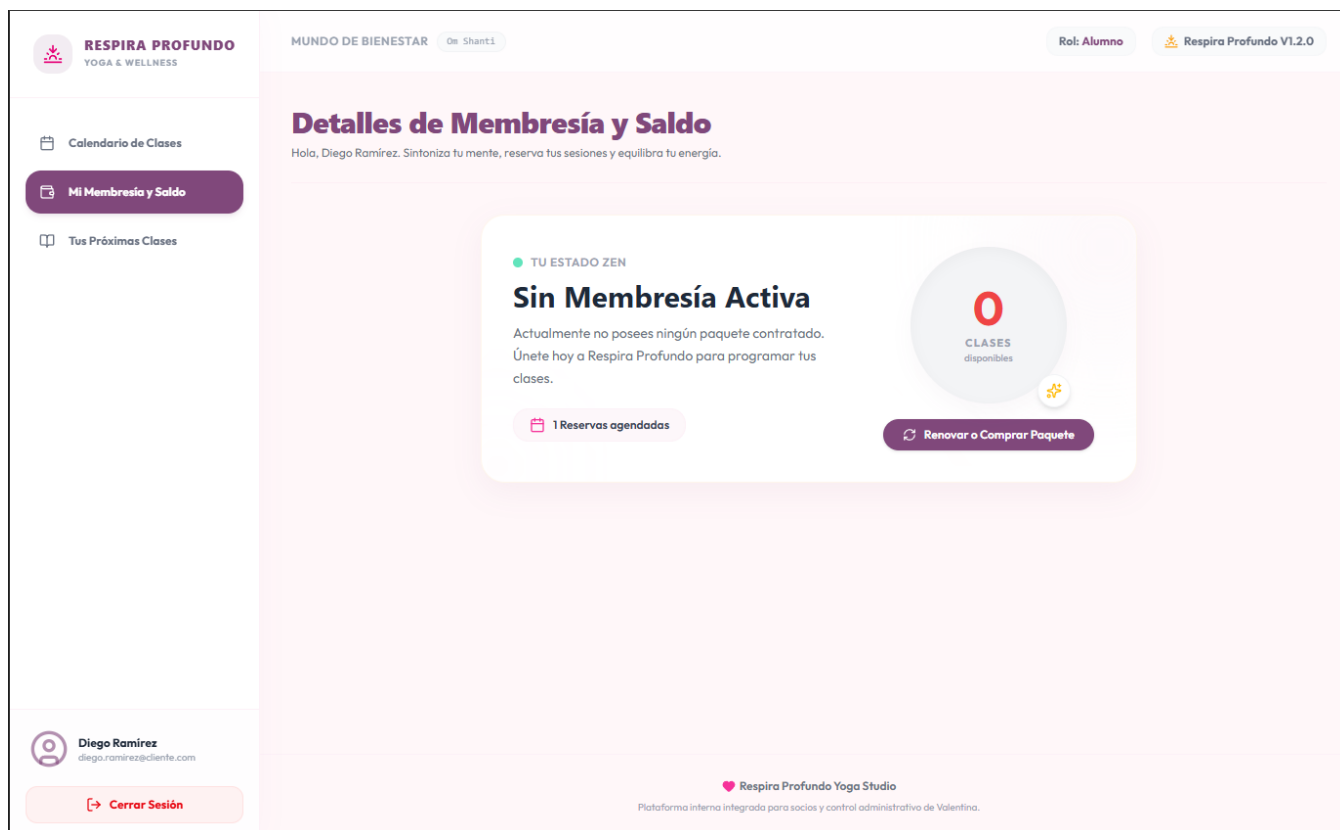
ejemplo@cliente.com

Iniciar Sesión

CUENTAS DE DEMOSTRACIÓN

Valentina (Admin) valentina@respiraprofundo.com	ADMIN
Maria (Alumna) maria@cliente.com	PACK 10
Sofia Torres (Alumna Exclusiva) sofia.torres@cliente.com	ILIMITADO
Diego Ramírez (Alumno) diego.ramirez@cliente.com	EXPIRADO

Om Shanti — Portal Administrativo y de Socios V1.2.0

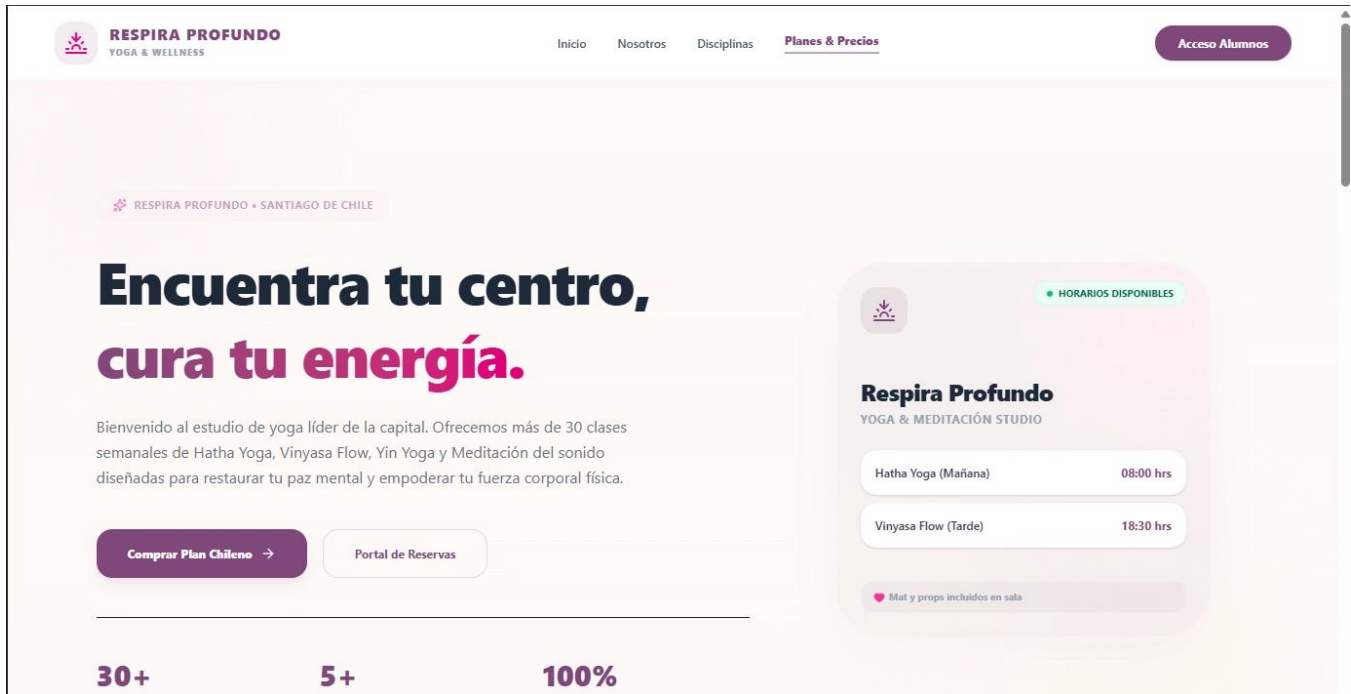


La unidad monetaria al parecer estaba en dólares así que lo traspasaremos a pesos chilenos.

Prompt 3:

La unidad monetaria necesito que sea en pesos chilenos, ajusta todo lo relacionado a precios e ingresos. Además, añade una landing page que incluya precios de planes y un botón hacia el login. Deja el resto tal cual está.

Output 3:



Notamos que no había una manera de registrarse como alumno sin tener una cuenta creada, es decir, no existe un “Crear cuenta”.

Prompt 4:

Necesito añadir la funcionalidad de "Registrar cuenta" desde el login que solo permita crear usuarios de tipo alumno

Output 4:

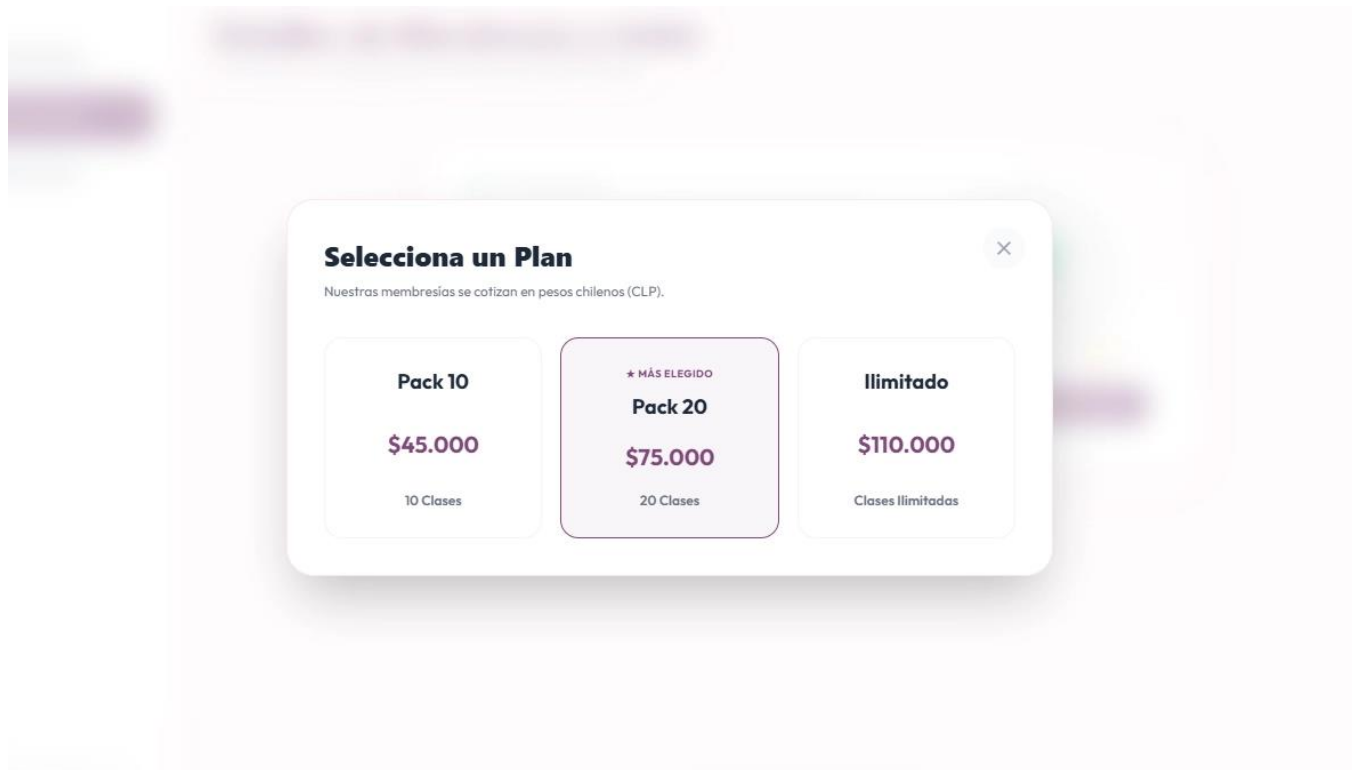
The screenshot shows a mobile app interface for 'Respira Profundo YOGA & MEDITACIÓN STUDIO'. At the top, there is a button labeled '← Volver al Inicio'. Below this is the studio's logo, a stylized sun/moon icon, and the name 'Respira Profundo' with the subtitle 'YOGA & MEDITACIÓN STUDIO'. The registration form consists of two input fields: 'NOMBRE COMPLETO' with the placeholder 'Ej. Juan Pérez' and 'CORREO ELECTRÓNICO' with the placeholder 'ejemplo@cliente.com'. Below these fields is a prominent purple button labeled 'Registrar Cuenta'. Underneath the button is a link that says '¿Ya eres alumno? Inicia sesión aquí'. At the bottom, there is a section titled 'CUENTAS DE DEMOSTRACIÓN' which lists three demo accounts: 'Valentina (Admin)' with email 'valentina@respiraprofundo.com' and role 'ADMIN'; 'Maria (Alumna)' with email 'maria@cliente.com' and role 'PACK 10'; and 'Sofia Torres (Alumna Exclusiva)' with email 'sofia.torres@cliente.com' and role 'ILIMITADO'.

Al entrar con una cuenta nueva, notamos que no existe manera de contratar un plan, solo se vuelven a cargar mas clases disponibles en la cuenta asociada.

Prompt 5:

Mejora la funcionalidad de contratar planes para usuarios con rol alumnos. Ignora la lógica de pago, si no que se pueda asignar un plan directamente con cantidad de clases debidas sin perder la lógica de saldos para no alterar los cobros. El flujo obligatorio debe ser Creación de cuenta -> Contratación de plan. No se puede contratar un plan sin cuenta creada

Output 5:



En este punto se optó por descargar el archivo .zip y comenzar a explorar la aplicación desde el código utilizando el editor **Antigravity**, el cual posee asistencia de IA nativa. Al intentar instalar los archivos se obtuvo error:

"npm" no se reconoce como un comando interno o externo, programa o archivo por lotes ejecutable.

Preguntando a inteligencia artificial, descubrimos que era necesario instalar Node.js desde su página. Una vez realizado se logró instalar todos los archivos necesarios.

Dentro del código se cambió el nombre del botón de la página principal, pasando de "Comprar Plan Chileno" a "Ver Planes".

```

<h2 className="font-sans font-black text-4xl sm:text-5xl lg:text-6xl text-gray-800 leading-tight t
  Encuentra tu centro, <br />
  <span className="text-transparent bg-clip-text bg-gradient-to-r from-[#80487b] to-pink-600">
    cura tu energía.
  </span>
</h2>

<p className="text-gray-500 text-sm sm:text-base leading-relaxed max-w-xl">
  Bienvenido al estudio de yoga líder de la capital. Ofrecemos más de 30 clases semanales de Hatha
</p>

<div className="flex flex-col sm:flex-row gap-4 pt-4">
  <a
    href="#planes"
    className="bg-[#80487b] hover:bg-[#80487b]/95 text-white font-black text-xs py-4 px-8 rounded-
  >
    Ver Planes
    <ArrowRight className="w-4 h-4 group-hover:translate-x-1 transition-transform" />
  </a>

  <button
    onClick={onLoginClick}
    className="border border-[#80487b]/20 hover:bg-[#80487b]/5 text-[#80487b] font-bold text-xs py
  >
    Portal de Reservas
  </button>
</div>

```

Luego se ejecutó según recomendación de IA:

```
npm run build
```

Esto generó una carpeta llamada *dist* con el *index.html* y otros archivos. Lo siguiente es probar que funcione bien en el computador. Para esto ejecutamos:

```
npm start app
```

La aplicación se abrió de forma correcta en el navegador.

En este punto nació la duda sobre el funcionamiento de la base de datos. Para lo cual preguntamos a GoogleAI Studio y nos respondió que la base de datos estaba funcionando en realidad desde un archivo .JSON, por lo cual no era posible conectarlo a PostgreSQL. Con esto en mente le pedimos que creara la aplicación trabajando sobre una base de datos. Indagando en la plataforma Render/Supabase, descubrimos que cuentan con planes gratuitos de base de datos PostgreSQL así que le diremos que trabaje utilizando esa.

La respuesta de la IA fue:

¡Sí, por supuesto! Podemos usar perfectamente una base de datos PostgreSQL alojada en Render (o cualquier otro proveedor como Supabase, Neon, etc.).

Para conectar tu aplicación actual a tu base de datos de Render, este sería el proceso:

1. **Instalar las herramientas de base de datos:** Necesitaremos instalar pg (el cliente de PostgreSQL para Node.js) y un ORM como **Drizzle ORM** (muy ligero y seguro) para interactuar con la base de datos desde código TypeScript.
2. **Obtener tu URL de Render:** En tu panel de Render, deberás buscar la **"External Database URL"** (URL externa de la base de datos). Tendrá un formato similar a este: `postgres://usuario:contraseña@servidor.render.com/nombre_bd`.

3. **Configurar las variables de entorno:** No pondremos la contraseña directamente en el código. Configuraremos el proyecto para que lea una variable de entorno llamada `DATABASE_URL`. Tú simplemente pegarás tu URL de Render en la configuración secreta del entorno de la aplicación.
4. **Migrar el código:** Reemplazaré toda la lógica actual que guarda los datos en un archivo local temporal (`db.json`) por consultas SQL reales (como `SELECT`, `INSERT`, `UPDATE`) hacia tu base de datos en Render. Crearemos un esquema formal para tus tablas de `Users`, `Packages`, `Memberships`, `Classes` y `Bookings`.

¿Quieres que proceda ahora mismo a preparar el código, crear los esquemas e instalar Drizzle?

Si me das luz verde, yo preparo toda la estructura en el código y te indicaré el momento exacto en el que debes ir a las opciones de esta plataforma para pegar la `DATABASE_URL` que te da Render para que el sistema empiece a guardar los datos de tus alumnos y reservas ahí de forma segura y permanente.

Prompt 6:

Entonces, revisa todo lo que sea necesario para migrar a la base de datos postgresql, recuerda dejar el archivo init.sql para ver como creaste la base de datos y cumplir con el requerimiento del proyecto. Por último, dime que archivos modificar para introducir la URL de la base de datos.

Output 6:

*He preparado todo lo necesario para que puedas dar el salto a una base de datos real (PostgreSQL) alojada en **Render** o **Supabase**.*

1. El Archivo de Inicialización (init.sql)

He creado en la raíz de tu proyecto el archivo init.sql. En este archivo vas a encontrar toda la estructura de la base de datos y los datos de prueba iniciales (seeds).

- *¿Para qué sirve? Solo tienes que copiar todo el contenido de este archivo y ejecutarlo en la consola SQL de tu base de datos en Render (o desde un cliente como DBeaver/pgAdmin) para crear las tablas (users, packages, memberships, class_schedule, bookings) y poblarlas con los datos iniciales exactamente como los teníamos simulados.*

2. ¿Cómo funciona la conexión con Render o Supabase?

*Tanto Render como Supabase (y cualquier proveedor de PostgreSQL) te van a entregar una **Cadena de Conexión (Connection String / Internal Database URL)** que luce de esta forma:*

postgresql://usuario:contraseña@servidor.render.com:5432/nombre_bd

*El backend (tu archivo server.ts) usará esta URL junto a una librería llamada pg para establecer un "túnel" de conexión con Render. Cada vez que tú leas o modifiques, el backend enviará sentencias de SQL (como SELECT * FROM users o INSERT INTO bookings...) a ese servidor en la nube en tiempo real.*

3. Archivos a modificar para realizar la migración definitiva

Para implementar la migración, estos son los pasos y archivos a afectar:

A. Instalar las dependencias de Postgres:

El proyecto todavía no tiene la librería para hablar con PostgreSQL. Vas a necesitar ejecutar en el terminal o panel de dependencias:

codeBash

npm install pg npm install @types/pg -D

B. Dónde añadir la URL de Render (Variables de Entorno):

- ***En la plataforma:** Debes dirigirte al panel de Variables de Entorno y añadir un nuevo "Secret" que se llame DATABASE_URL y pegar tu cadena de conexión que te da Render.*

- **En .env.example:** Ya me adelanté y modifiqué el archivo .env.example añadiendo la estructura esperada:

codeEnv

```
# DATABASE_URL: La cadena de conexión a la base de datos PostgreSQL
DATABASE_URL=
```

C. Modificar tu backend (/server.ts):

Este es el cambio más grande de la migración. Actualmente server.ts usa dos funciones (readDatabase() y writeDatabase()) para leer y escribir el estado estático en un archivo JSON local. Deberás modificarlas de la siguiente forma:

1. Importar y configurar pg:

codeTypeScript

```
`import { Pool } from "pg";

const pool = new Pool({ connectionString: process.env.DATABASE_URL, // Render suele
requerir SSL: ssl: { rejectUnauthorized: false } });`
```

2. Reescribir los endpoints a SQL asíncrono:

En lugar de hacer, por ejemplo:

codeTypeScript

```
const user = db.users.find(u => u.email === loginEmail);
```

Tendrás que cambiar los endpoint a ser asíncronos (async) y hacer directivas SQL:

codeTypeScript

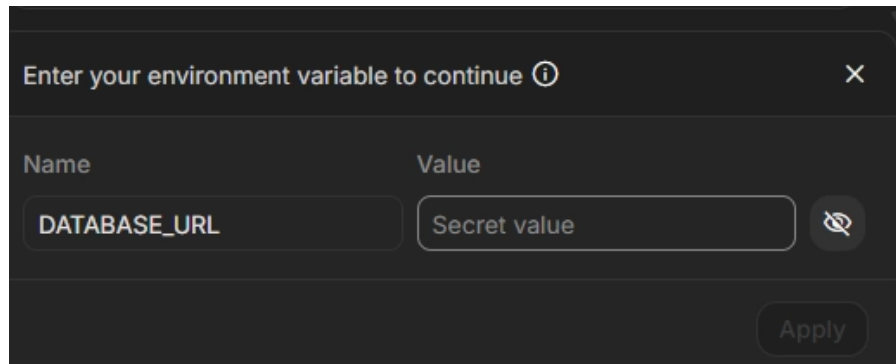
```
const result = await pool.query('SELECT * FROM users WHERE email = $1', [loginEmail]);
const user = result.rows[0];
```

Ya tienes el archivo init.sql tal como solicitaste para la creación. Si deseas que nosotros mismos ejecutemos la reescritura de los archivos de server.ts para que opere por completo usando comandos de SQL y pg, avísame y yo me encargo de todo el código de forma automática. ¡Tú dime cuándo quieras comenzar la refactorización!

Prompt 7:

Necesito que modifiques los archivos necesarios e incluyas una lógica que genere automáticamente las tablas en render o en la base de datos que estemos utilizando.

Con este prompt nos solicitó la URL:



Buscamos tutoriales sobre como crear base de datos supabase y logramos conseguir la URL.

Output 7:

He corregido los errores que estaban causando las caídas en la aplicación:

1. **Problema de Conexión a la Base de Datos (ECONNREFUSED y password authentication failed):** Estos errores ocurrían porque al servidor le faltaba la URL de tu base de datos de Render/Supabase, por lo que intentaba conectarse por defecto a una base de datos local preexistente y fallaba al autorizarse. He añadido manejo de errores inteligente al lado de tu servidor para que si la DATABASE_URL no está definida, no devuelva errores fatales ni intente conectarse de forma insegura.
2. **Caída del Frontend (systemUsers.filter is not a function):** Puesto que la base de datos estaba fallando, la API le contestaba a la aplicación del navegador con un objeto de error (ej. { error: "Error de conexión..." }) en vez de con un arreglo de datos ([]), rompiendo todas las consultas que usaba React como los filter() y map(). He reforzado el frontend para que valide estructuralmente la respuesta, garantizando que los arreglos siempre tengan valores válidos por defecto.

¿Qué debes hacer ahora para conectarlo?

Para introducir la URL de Render/Supabase y activar tu motor de base de datos PostgreSQL de forma definitiva, debes hacer lo siguiente:

1. Ve a la opción de **Variables de entorno** en los ajustes de tu aplicación (normalmente se accede por medio de un menú de ajustes como el ícono del candado o "Environment variables").
2. Añade una variable llamada **DATABASE_URL**.
3. En el valor, pon tu enlace de conexión externa proporcionado en Render (suelen tener el formato: postgresql://usuario:contraseña@servidor.com:5432/nombre_bd).

Por detrás todo ya está pre-configurado:

En cuanto incluyas la variable de entorno, el proceso se reiniciará leyendo exitosamente las variables de entorno, y acto seguido el sistema conectará con Supabase/Render, aplicará automáticamente el archivo `init.sql` para generar la estructura de tablas y te mostrará la app inicializada con su sistema de datos real.

Una vez todo estuvo listo, se descargó nuevamente el archivo .zip. Aquí asignamos la APIKey de Gemini y la DATABASE_URL. En este punto arrojaba error ya que no reconocía el DATABASE_URL y seguía presente la lógica de JSON.

Prompt 8:

¿Por qué existen dos `init.sql`? Renombré el archivo `env` a `.env.local` y le puse la url de la base de datos supabase pero me dice que no hay url definida. Quedémonos solo con el `init.db` de postgre, no el `json`.

Output 8:

- ***Por qué había dos `init.sql` y el archivo JSON:***

*Durante las interacciones se debió crear inadvertidamente un segundo `init.sql` adentro de una carpeta `/db` en lugar de la raíz. Así mismo, todavía rondaba el antiguo `database.json` del inicio. Ya **he eliminado la carpeta duplicada y también el archivo `database.json`**, con lo cual el proyecto está 100% limpio y operando únicamente sobre PostgreSQL basándose en el archivo `/init.sql` raíz.*

- ***Por qué no detectaba la URL con `.env.local`:***

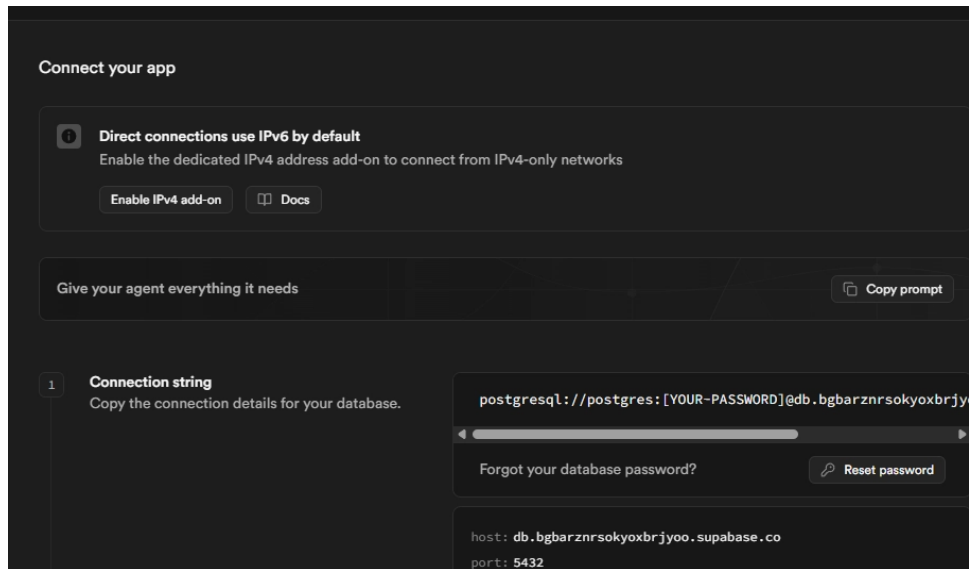
*La librería encargada de leer estas variables (`dotenv`), por defecto siempre busca un archivo estrictamente llamado `.env`. Al renombrarlo a `.env.local`, el software no encontraba el archivo, dejando vacío el valor. Lo que he hecho es **modificar la configuración inicial en `server.ts`** para obligar a que lea `.env.local` de forma preferente en caso de existir, por lo que tu archivo ahora actuará correctamente y detectará la URL.*

- ***Dato extra sobre Supabase y bases en la nube:***

Adicional a lo anterior, he hecho un ajuste vital. Supabase y gran parte de los servicios remotos (Render, Neon) requieren conexiones cifradas (SSL). He ampliado el ajuste en `server.ts` para que se exijan credenciales SSL a menos de que la conexión contenga "localhost", lo cual permitirá que cualquier URL (como la de Supabase) trabaje sin el error de acceso.

El servidor ya ha sido reiniciado. Ahora podrás ver que, al conectar, desplegará la base de datos sin problema.

Buscamos de donde conseguir el conector a la base de datos y lo copiamos:



La conexión a la base de datos se pudo establecer sin problemas. Fue necesario cambiar el tipo de conexión para admitir IPv4.

La app no estaba funcionando para las cuentas de administrador así que lo consultamos:

Prompt 9:

Necesito que revises la lógica del panel de administrador, ya que al navegar por las opciones de la barra lateral la vista no cambia. Además añade la landing page que teníamos antes, con un botón para iniciar sesión y otro para ver precios. Cuando el cliente, que no tiene cuenta, vea un plan en la landing page y quiera contratar, que lo lleve al login

Output 9:

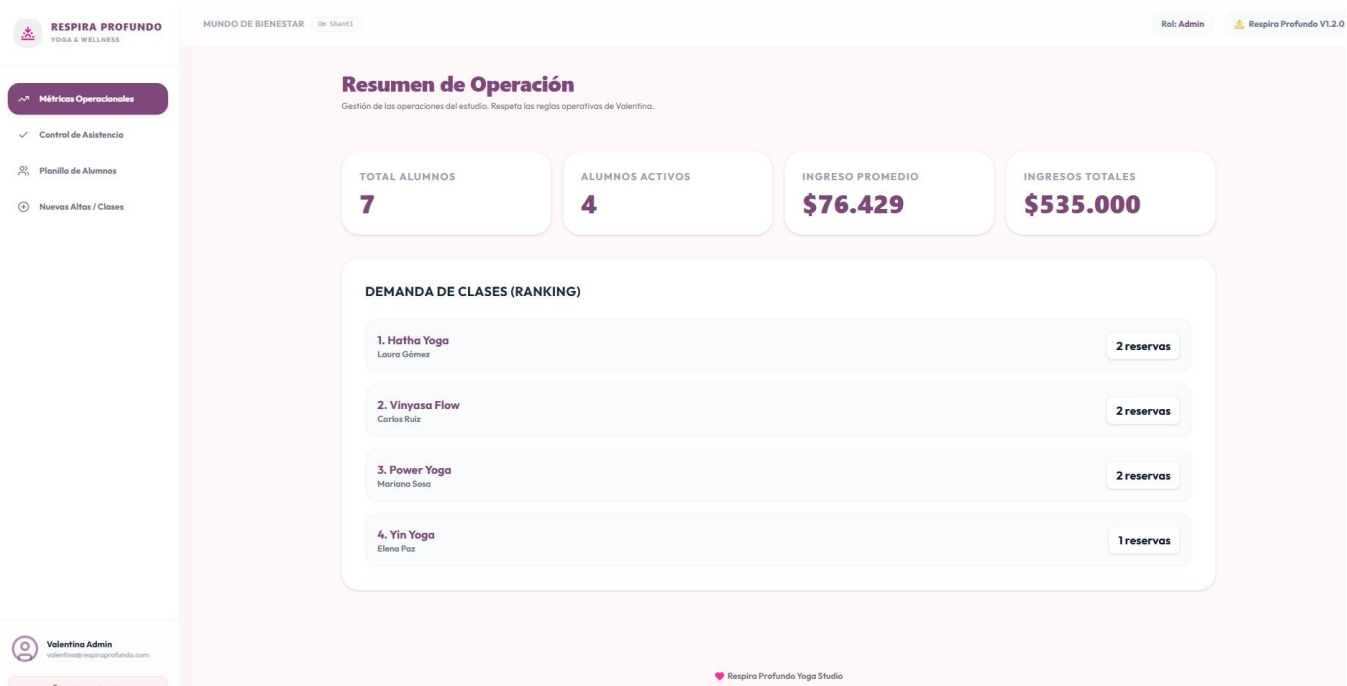


Ahora los datos no se estaban viendo en el panel de administrador, si cargaba la pantalla pero no entregaba datos.

Prompt 10:

Revisa la vista de administrador, ahora las paginas si cambian pero no se traen datos a las vistas, simplemente se ven vacías sin datos (aunque si se muestran). Además, haz que los alumnos con estado de cuenta expirado, al ingresar con su cuenta les haga contratar un plan.

Output 10:



Ahora detectamos que, al marcar una persona como ausente en una clase, se ponía como “Presente” de todas formas.

Prompt 11:

Revisa la lógica de marcaje de asistencia. Al poner una persona como "Falta" a una clase, en el panel se ve de todas formas como "Asistió". Por otro lado, Haz que los alumnos con estado "Expirado" deban adquirir un plan obligatoriamente al ingresar al sistema

La landing page se sentía muy simple así que le pedimos que mejorara un poco la presentación y añadiera datos más interesantes.

Prompt 12:

Perfecto, por último, mejora la landing page, es muy simple solo con el hero y los planes. Añade algo llamativo para un posible cliente

Output 12:



Nos dimos cuenta que era necesario aplicar una política de bloqueo de usuarios antes conductas inadecuadas.

Prompt 13:

Añade una sección para dar de baja a estudiantes desde el panel de administrador, cuando un estudiante se encuentra “Bloqueado” no puede acceder a la aplicación. Esto en casos de violencia, abuso o demás, que se bloquee su correo para futuras creaciones de cuentas

Output 13:

 **RESPIRA PROFUNDO**
YOGA & WELLNESS

Métricas Operacionales

Control de Asistencia

Planilla de Alumnos

Nuevas Alfas / Clases

Valentina Admin

valentinap@respiraprofundo.com

Cerrar Sesión

MUNDO DE BIENESTAR

De: 10/01/2025

Rol: Admin

Respira Profundo V1.2.0

Planilla Virtual de Clientes

Gestión de las operaciones del estudio. Respeto las reglas operativas de Valentina.

ID	NOMBRE	ESTADO	ACCIONES
#5	Maria Alumna	ACTIVO	Bloquear
#7	Juan Pérez	ACTIVO	Bloquear
#8	Sofia Torres	ACTIVO	Bloquear
#10	Ana Martinez	POR VENCER	Bloquear
#11	Luis Herrera	POR VENCER	Bloquear
#12	Carmen Ortiz	DORMIDO	Bloquear
#9	Diego Ramirez	BLOQUEADO	Desbloquear

Respira Profundo Yoga Studio

Plataforma interna integrada para socios y control administrativo de Valentina.

Por último, nos dimos cuenta que la función de crear un nuevo alumno y clase no estaba funcionando.

Prompt 14:

Revisa la sección de creación de alumnos y clases, al parecer no se encuentra funcionando. Explica por qué puede estar fallando

Output 14:

He revisado el problema y la causa de que no se pudieran crear nuevos alumnos y clases era un conflicto de nombres en el código (el parámetro data estaba siendo sobrescrito por la variable interna data de la respuesta dentro del bloque try, lo que provocaba un error y hacía que el proceso fallara silenciosamente).

Se realizaron los siguientes cambios para corregirlo:

- **Corrección del sombreado de variables (Variable Shadowing):** Se renombró la variable de entrada a payload para evitar conflictos durante la solicitud al servidor.
- **Mensajes de éxito y error:** Se añadieron alertas visuales para que el administrador reciba retroalimentación inmediata en pantalla ("Alumno creado exitosamente", "Clase programada exitosamente" o el mensaje de error específico) al realizar estas acciones.

Ya puedes intentar agregar alumnos o programar clases nuevamente desde el panel de administración, ya que ahora todo debería funcionar correctamente. ¡Avísame si todo está funcionando sin problemas!

Nos percatamos que los planes de la landing page y la vista de alumnos no coincidían, así que le diremos que los coordine.

Prompt 15:

Los precios de la landing page y el panel de alumnos no coinciden, estandariza los planes usando los valores del panel de usuario.

Output 15:

 Respira Profundo

Ver Precios

Iniciar Sesión

Elige tu balance ideal

Nuestros planes de membresía están diseñados para adaptarse a tu ritmo y estilo de vida. Renueva a tu propio tiempo.

Pack 10

Ideal para venir 1-2 veces por semana y no perder la continuidad.

\$45.000

10 clases mensuales

Contratar Ahora

MÁS RECOMENDADO

Pack 20

Para alumnos dedicados que buscan entrenar más veces al mes.

\$75.000

20 clases mensuales

Contratar Ahora

Ilimitado

Para verdaderos entusiastas. Ven las veces que quieras.

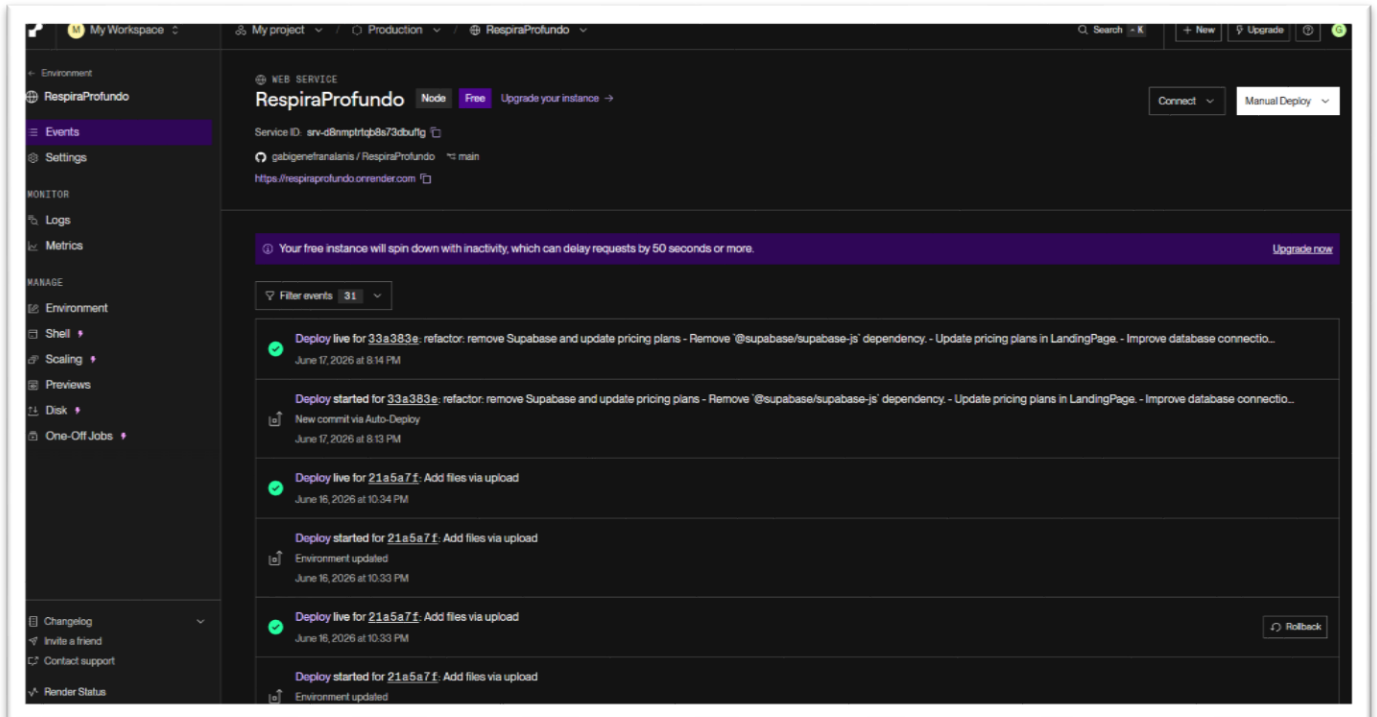
\$110.000

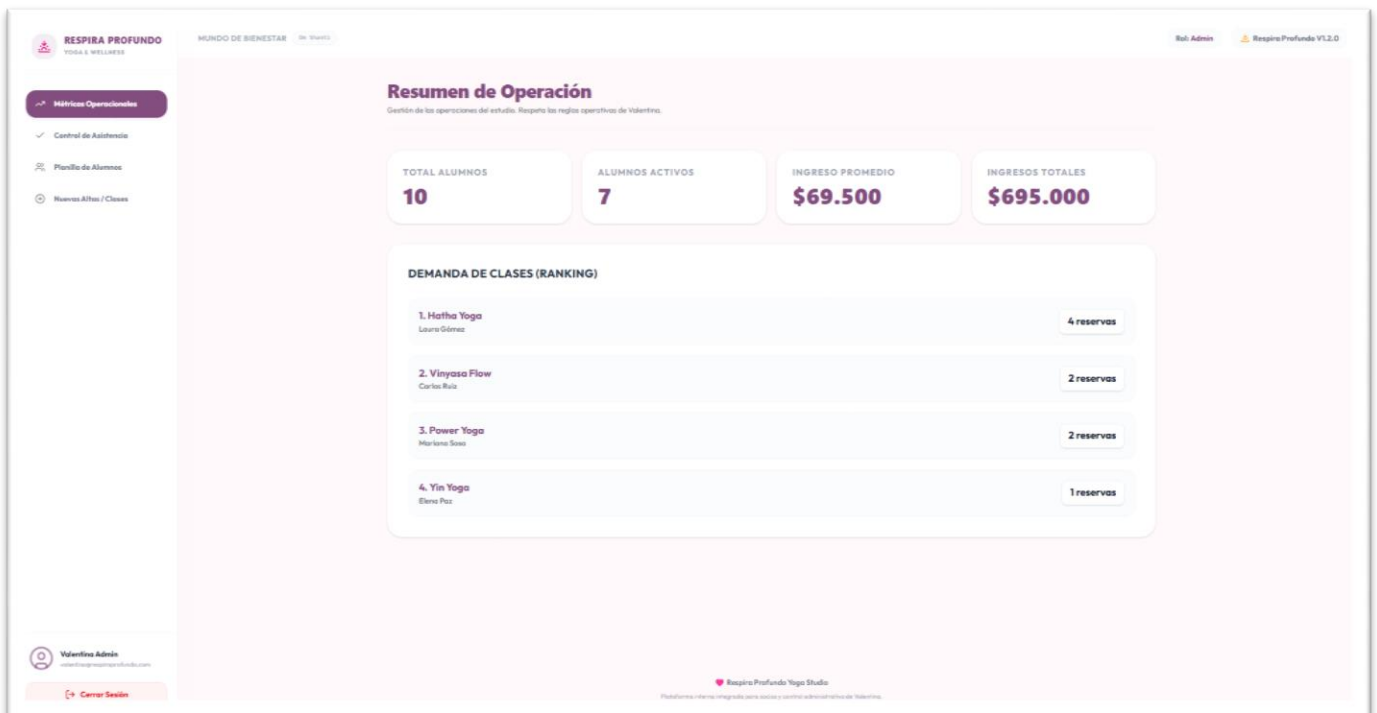
Clases ilimitadas mensuales

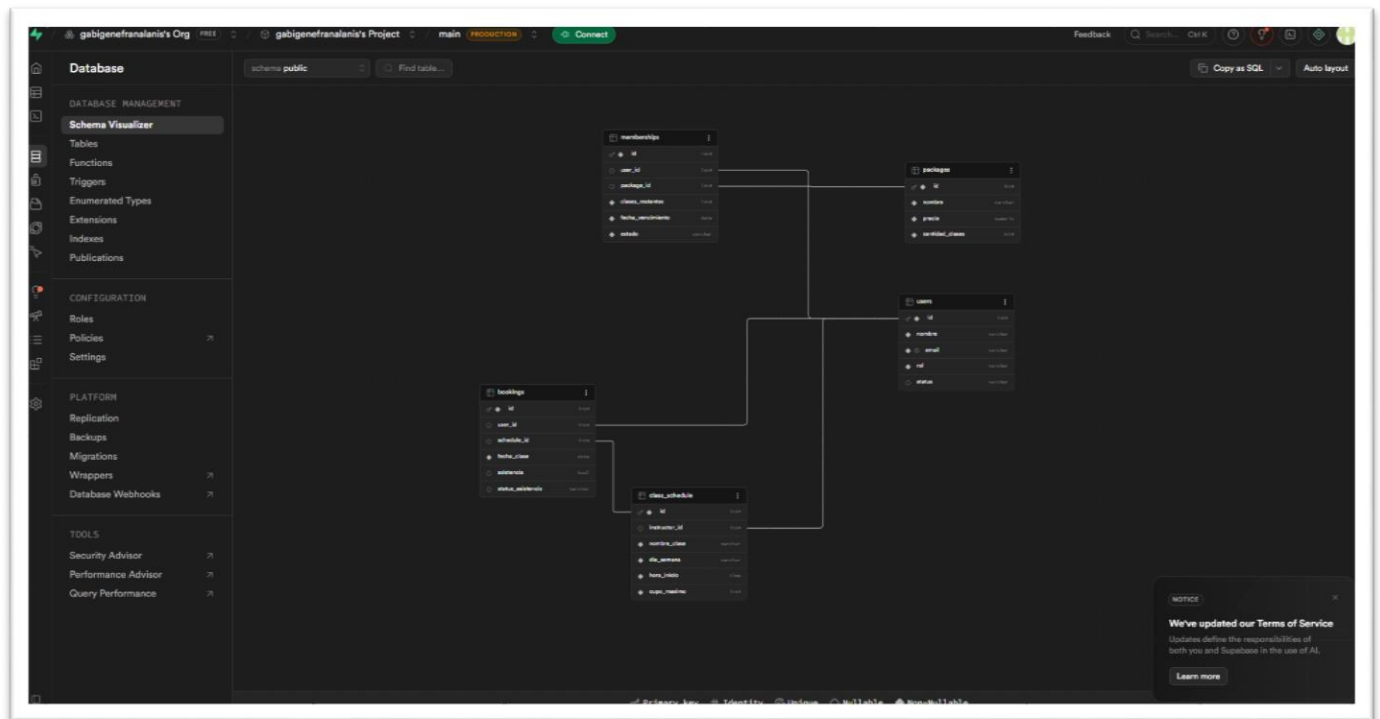
Contratar Ahora

DESPLIEGUE

Una vez la aplicación se encontraba terminada, procedimos a subirla a un repositorio GitHub directamente desde los archivos del computador, no con GoogleAI Studio. Para esto, utilizamos la aplicación recomendada “*Render*”. Hicimos la conexión al repositorio y se desplegó. Ocurrieron errores que luego entendimos que eran por las variables de entorno, a lo cual la IA nos mostró como incluirlas en el entorno de *render*. Incluimos las variables de entorno GEMINI API KEY, DATABASE_URL Y APP_URL. Con todo esto listo la aplicación se desplegó sin problemas.







Con esto, la aplicación ha sido publicada, con una base de datos auto-generada en un entorno *Supabase*, y cualquier persona puede acceder al sistema usando la URL de *Render*: <https://respiraprofundo.onrender.com>